

EXPERIMENT 2

Run, Test and Manage a Docker Container

Objective

To learn how to use an existing Docker image to create a container, check whether the container is running, test the application, view logs, inspect the container, enter the container, stop and restart it, and finally remove it.

Learning Outcomes

- Use an existing Docker image to create a container.
- Check whether a container is running.
- Test the application running inside the container.
- View container logs and basic container information.
- Stop, start, inspect, enter, and remove a container.
- Understand the basic Docker container lifecycle.

Requirements

- Docker Desktop installed and running.
- PowerShell.
- The Docker image my-python-app created in Experiment 1.
- Internet connection is not required when the image already exists locally.

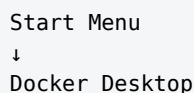
Important: In this experiment we are not rebuilding the application image. We use the image created in Experiment 1 and focus on what can be done with a container after it has been created.

PART 1 - START DOCKER

Step 1: Open Docker Desktop

Open the Windows Start menu, search for Docker Desktop, and open it. Wait until Docker Desktop finishes starting.

Open from Start Menu



```
Start Menu
↓
Docker Desktop
```

- Docker Desktop provides the Docker environment on the Windows computer.
- PowerShell is used to enter Docker commands.
- Docker should be running before students create or manage containers.

Step 1 continued: Check Docker from PowerShell

Open PowerShell and verify that Docker is available.

PowerShell command

```
docker --version
```

Expected result

```
Docker version 29.x.x
```

- The command checks whether the Docker CLI is available.
- The version shown may be different on different computers.
- A valid version confirms that PowerShell can communicate with Docker.

PART 2 - CHECK THE EXISTING IMAGE

Step 2: Display Docker images

The image my-python-app was created in Experiment 1. We will confirm that it is still stored on the computer.

Run in PowerShell

```
docker images
```

Expected result

```
REPOSITORY TAG IMAGE ID
my-python-app latest af5ed65648bb
```

- my-python-app is the image created during Experiment 1.
- We do not need to build the image again for this experiment.
- An image is a reusable template from which containers can be created.
- The image can be reused to create multiple containers.

PART 3 - CREATE AND RUN A NEW CONTAINER

In Experiment 1, the application container was named my-python-container. In Experiment 2 we use a different name so students can clearly see that a new container is being created from the same image.

Step 3: Create and run the container

Run the following command in PowerShell.

Run in PowerShell

```
docker run -d -p 5001:5000 --name test-python-container my-python-app
```

Expected result

```
A long value such as:
d8b7c1.....
```

- Docker creates a new container from the existing my-python-app image.

- The new container is named test-python-container.
- -d runs the container in the background.
- -p 5001:5000 maps Windows port 5001 to container port 5000.

Port mapping

```
Windows Host
Port 5001
|
↓
Docker Container
Port 5000
|
↓
Flask Application
```

We use host port 5001 in this experiment so it is separate from the port used in Experiment 1.

PART 4 - CHECK THE RUNNING CONTAINER

Step 4: Use docker ps

Run the command below to display the Docker containers that are currently running.

Run in PowerShell

```
docker ps
```

Expected result

```
CONTAINER ID IMAGE STATUS PORTS
xxxxxxx my-python-app Up ... 0.0.0.0:5001->5000/tcp
```

- docker ps displays only running containers.
- my-python-app is the image used by the container.
- test-python-container is the container created in this experiment.
- Up indicates that the container is currently running.

PART 5 - TEST THE APPLICATION

Step 5: Open the application in a browser

Open Google Chrome or Microsoft Edge and enter the following address.

Browser address

```
http://localhost:5001
```

Expected result

```
Hello! My first Docker application is running.
```

- The browser sends a request to host port 5001.
- Docker forwards the request to port 5000 inside the container.

- Flask receives the request and processes the root route.
- The response is returned to the browser.

Request flow

```
Browser
↓
localhost:5001
↓
Windows Host
↓
Docker Port Mapping
↓
Container Port 5000
↓
Flask Application
↓
Response
↓
Browser
```

PART 6 - TEST THE CONTAINER USING POWERSHELL

Step 6: Test without opening a browser

PowerShell can send an HTTP request directly to the running application.

Send an HTTP request

```
Invoke-WebRequest http://localhost:5001
```

Display only the returned content

```
(Invoke-WebRequest http://localhost:5001).Content
```

Expected result

```
Hello! My first Docker application is running.
```

- This verifies that the web application responds to an HTTP request.
- The browser is not required for this test.
- PowerShell communicates with the application through the published host port.

PART 7 - VIEW CONTAINER LOGS

Step 7: Display logs

Use Docker logs to view messages generated by the application running inside the container.

Run in PowerShell

```
docker logs test-python-container
```

Expected result

```
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.x.x.x:5000
```

```
GET / HTTP/1.1
```

- Logs show whether the Flask application started correctly.
- Logs can show requests received by the application.
- Logs are useful for finding application startup and runtime problems.

PART 8 - INSPECT THE CONTAINER

Step 8: Display container information

Run the inspect command. It produces detailed information, so students only need to identify a few important sections.

Run in PowerShell

```
docker inspect test-python-container
```

- The inspect command gives detailed information about the container.
- Important information includes the container name, image, state, ports, and network.
- This information is useful during troubleshooting.
- Students do not need to understand every field returned by the command.

Look for

```
Name
Image
State
Ports
Network
```

PART 9 - ENTER THE RUNNING CONTAINER

Step 9: Open a shell inside the container

Because the application is running inside the container, we can open a shell and look at the environment from inside.

Run in PowerShell

```
docker exec -it test-python-container /bin/sh
```

Expected result

```
/app #
```

- docker exec executes a command inside a running container.
- -it gives an interactive terminal session.
- /bin/sh opens a shell inside the container.

- The prompt changes because you are now working inside the container.

PART 10 - EXPLORE THE CONTAINER

Step 10: List files inside the container

While you are inside the container, type the command below.

Run inside the container

```
ls
```

Expected result

```
app.py
requirements.txt
```

- The application file `app.py` exists inside the container.
- The dependency file `requirements.txt` also exists inside the container.
- These files were copied into the image during the image build.
- The container was created from that image.

Step 11: Check the current directory

Still inside the container, check the current working directory.

Run inside the container

```
pwd
```

Expected result

```
/app
```

- The current directory is `/app`.
- This comes from the Dockerfile instruction `WORKDIR /app`.
- Docker starts commands from this working directory.

Dockerfile instruction

```
WORKDIR /app
```

PART 11 - EXIT THE CONTAINER SHELL

Step 12: Exit the container shell

Type `exit` to leave the shell and return to PowerShell on the Windows host.

Run inside the container

```
exit
```

Expected result

```
PS C:\Users\Admin>
```

- The exit command leaves the interactive shell.
- It does not delete the container.
- The container can continue running after the shell is closed.
- You are only leaving the interactive session.

Check the container after exiting

```
docker ps
```

The container test-python-container should still be listed as running.

PART 12 - STOP THE CONTAINER

Step 13: Stop the running container

Stop the container without deleting it.

Stop the container

```
docker stop test-python-container
```

Check running containers

```
docker ps
```

- docker stop stops a running container.
- The stopped container still exists.
- Because it is stopped, it no longer appears in docker ps.
- Stopping a container is different from removing a container.

PART 13 - CHECK STOPPED CONTAINERS

Step 14: Display all containers

Use docker ps -a to display both running and stopped containers.

Run in PowerShell

```
docker ps -a
```

Expected result

```
CONTAINER ID IMAGE STATUS
xxxxxxx my-python-app Exited ...
```

- docker ps shows only running containers.
- docker ps -a shows running and stopped containers.
- This lets students find a container even after it has stopped.

PART 14 - START THE SAME CONTAINER AGAIN

Step 15: Start the stopped container

Start the existing container again instead of creating a new one.

Start the container

```
docker start test-python-container
```

Check running containers

```
docker ps
```

Expected result

```
STATUS
Up ...
```

- docker start starts the existing stopped container.
- A new container is not created.
- The same container name and container identity are retained.
- The application becomes available again on host port 5001.

Open the browser again:

```
http://localhost:5001
```

Container lifecycle

```
Container
↓
STOP
↓
Stopped Container
↓
START
↓
Running Container
```

Important: The container is started again; it is not recreated.

PART 15 - CHECK THE CONTAINER ID

Step 16: Compare the Container ID

Run docker ps after restarting the container and compare the Container ID with the ID seen when the container was first created.

Run in PowerShell

```
docker ps
```

- The same container ID is associated with the same container.
- docker start does not create a new container.

- The container ID remains linked to that container.

Example

Original Container ID: d8b7c1...

After docker start:

Container ID: d8b7c1...

Same container.

PART 16 - REMOVE THE CONTAINER

Step 17: Stop and remove the container

Before removing the container, stop it if it is running. Then remove it.

Stop the container

```
docker stop test-python-container
```

Remove the container

```
docker rm test-python-container
```

Verify removal

```
docker ps -a
```

- docker rm removes the container.
- The removed container no longer appears in docker ps -a.
- Removing a container does not remove the image used to create it.
- The image my-python-app should still be available.

PART 17 - CONFIRM THE IMAGE STILL EXISTS

Step 18: Check the image

List Docker images after deleting the container.

Run in PowerShell

```
docker images
```

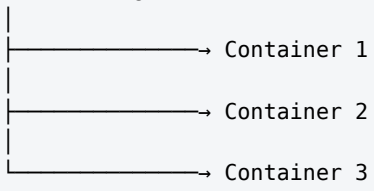
Expected result

```
my-python-app
```

- The container has been removed.
- The Docker image remains on the computer.
- The same image can be used to create another container later.
- One image can be used as the source for multiple containers.

Image reuse concept

Docker Image



COMPLETE EXPERIMENT 2 FLOW

The complete experiment can be remembered as a container lifecycle.

```
Existing Docker Image
|
↓
my-python-app
|
↓
docker run
|
↓
test-python-container
|
↓
docker ps
|
↓
Test Application
|
↓
localhost:5001
|
↓
docker logs
|
↓
docker inspect
|
↓
docker exec
|
↓
Enter Container
|
↓
docker stop
|
↓
Stopped Container
|
↓
docker start
|
↓
Running Container
|
↓
docker rm
|
↓
Container Removed
```

IMAGE VS CONTAINER

```
Docker Image
my-python-app
↓
↓ docker run
↓
Docker Container
test-python-container
```

An image is the reusable package/template. A container is an instance created from that image.

PROJECT DESCRIPTION - STEP-BY-STEP UNDERSTANDING

- 1. Start from the existing image: The experiment begins with the Docker image my-python-app that was created in Experiment 1. Students do not rebuild it. This shows that a Docker image is reusable.
- 2. Create a container: The docker run command creates a new container from the image and starts it. The container is given the name test-python-container so it can be managed with simple commands.
- 3. Publish a port: The command uses -p 5001:5000. Port 5001 is the host-side port, while port 5000 is the port used by Flask inside the container. This makes the application reachable from the Windows host.
- 4. Check the running state: The docker ps command is used to see running containers. The status Up indicates that the container is running.
- 5. Test the application: Students access <http://localhost:5001>. The request reaches the host port, is forwarded by Docker to port 5000 in the container, and is processed by the Flask application.
- 6. Test using PowerShell: Invoke-WebRequest provides another way to test the HTTP response. This is useful when students want to verify the service without using a browser.
- 7. View logs: docker logs displays application output and request information. Logs are especially useful when a container starts but the application is not behaving as expected.
- 8. Inspect the container: docker inspect provides detailed metadata and configuration, including the container name, image, state, port configuration, and network information.
- 9. Enter the container: docker exec -it ... /bin/sh opens a shell inside the running container. This demonstrates that the application files and runtime environment are actually available inside the container.
- 10. Explore the container: ls shows the copied application files, while pwd shows the working directory. The working directory is /app because the Dockerfile specified WORKDIR /app.
- 11. Exit without deleting: The exit command only closes the interactive shell. The container may continue running, showing that leaving a shell session is different from stopping or removing the container.
- 12. Stop the container: docker stop changes the container from running to stopped. The container still exists and can later be started again.
- 13. View stopped containers: docker ps -a shows both running and stopped containers. This helps students understand that a stopped container is still a Docker object until it is removed.
- 14. Start the same container: docker start starts the existing container again. It does not create a new container, which is why the same container ID remains associated with it.

- 15. Remove the container: `docker rm` permanently removes the container. The image is left untouched, so the image can still be reused.
- 16. Overall concept: Experiment 2 focuses on the container lifecycle: run → test → inspect → enter → stop → start → remove. Students learn what they can do with a container after the image has already been created.

KEY COMMANDS USED

```
docker --version
docker images
docker run -d -p 5001:5000 --name test-python-container my-python-app
docker ps
Invoke-WebRequest http://localhost:5001
(Invoke-WebRequest http://localhost:5001).Content
docker logs test-python-container
docker inspect test-python-container
docker exec -it test-python-container /bin/sh
ls
pwd
exit
docker stop test-python-container
docker ps -a
docker start test-python-container
docker rm test-python-container
docker images
```

Expected final result: Students can create a container from the existing image, verify that the application is running, inspect and enter the container, stop and restart the same container, and finally remove the container while keeping the Docker image available.